

Shared by Swasthik. This is a copy of a chat between Claude and Swasthik. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

<https://claude.ai/share/fc281fae-3eee-4a82-9cd8-82cb0f4e8b25>, Can you go through this and help me complete the task 3 and task 4 step by step together?

Read a page

Read a page

I couldn't open that shared chat. The page loads as an empty shell for me, so I can't see the tasks in it, including Task 3 and Task 4.

Could you paste the text of Tasks 3 and 4 straight into this chat? If the earlier conversation set up context for them (a dataset, a project description, earlier answers), include the relevant parts too.

Once I can see them, we can go one task at a time. I'll start with Task 3, and we'll move to Task 4 when you're happy with it.

PDF

https://github.com/SETAPESU26/58_color_pattern, This is the github link and the project overview in the pdf. Can u help me to complete this project step by step together with you and me?

Yes, let's do it together. I read the repo README and your lab handout. This is a Pygame Simon-style memory game with one deliberate bug and three features to add.

The 4 tasks

Bug fix: each round should add exactly one new color, but the sequence currently explodes in length.

Acceleration: flash and pause durations shrink as rounds go up.

Sound: each of the four pads gets its own pitch, played on flash and on click.

Turn timer: a countdown bar during the player's turn, and game over if it runs out.

The handout wants a separate commit per task and a chat history export. The prompts you give me are part of what gets graded, so I'll keep each step small and clear.

Step 1: Setup (you do this)

Fork or clone the repo into your personal account. Don't open a PR to SETAPESU26.

git clone https://github.com/SETAPESU26/58_color_pattern

cd 58_color_pattern

pip install pygame

python main.py

Play a few rounds until the bug shows up, and record the 10-second "before" video. Get past round 2 on camera so the sequence duplication is visible.

Make a Lab-4 folder in your lab repo to collect the deliverables.

Step 2: Fix Task 1 (together)

Open game/game_engine.py and paste it here. If there's a game/color_button.py with relevant logic,

paste that too. I'll find the bug, explain why the sequence grows too fast, and give you a minimal fix. Then you commit it:

```
git add .  
git commit -m "Task 1: fix sequence duplication bug"
```

After that we'll do Tasks 2, 3 and 4 one at a time, each ending with its own commit. Task 3 (sound) is the only one that needs new tooling. We can generate tones with `pygame.mixer` and `numpy` or the `array` module, so no audio files are needed.

Tell me when the "before" video is recorded, and paste `game_engine.py` whenever you're ready.

Should I install `pygame` globally or in a `venv` or a folder?

Use a virtual environment (`venv`) inside the project folder. It keeps `pygame` (and anything else we add later, like `numpy` for the sound task) out of your other projects, and it's one command to set up.

From inside the cloned `58_color_pattern` folder:

Windows (PowerShell/CMD)

```
python -m venv venv  
venv\Scripts\activate  
pip install pygame  
python main.py
```

macOS / Linux

```
python3 -m venv venv  
source venv/bin/activate  
pip install pygame  
python main.py
```

Once it's activated, your prompt shows `(venv)`. You need to activate it again each time you open a new terminal.

Before your first commit, make sure the `venv` doesn't get pushed. Create a `.gitignore` file in the project root containing:

```
venv/  
__pycache__/
```

Otherwise `git add .` will try to commit thousands of `venv` files.

One caveat: if you're on Python 3.13 or newer and `pygame` fails to install, try `pip install pygame-ce`. It's a drop-in replacement and works the same with this code.

Let me know once the game runs and you've recorded the "before" video.

Designing a system for dynamically adjusting animation speed.

Task 2 only touches `game/game_engine.py`, in three places.

The idea

flash_duration and pause_duration are fixed numbers set once in __init__. We'll keep those as the starting speed and recalculate them at the start of every round. The rule is that each round shortens the flash and pause by a fixed amount, and a minimum keeps the game playable.

Change 1: __init__

Find these two lines:

```
python
self.flash_duration = 450
self.pause_duration = 200
```

Replace them with:

```
python
# Playback speed (milliseconds). Starts slow and speeds up each round.
self.base_flash_duration = 450
self.base_pause_duration = 200
self.min_flash_duration = 120
self.min_pause_duration = 60
self.flash_speedup_per_round = 35
self.pause_speedup_per_round = 15

self.flash_duration = self.base_flash_duration
self.pause_duration = self.base_pause_duration
```

Change 2: add a new method

Put it just above start_next_round:

```
python
def update_playback_speed(self):
    round_number = len(self.sequence) # round 1 has 1 step, round 2 has 2, ...
    steps_faster = round_number - 1

    self.flash_duration = max(
        self.min_flash_duration,
        self.base_flash_duration - steps_faster * self.flash_speedup_per_round,
    )
    self.pause_duration = max(
        self.min_pause_duration,
        self.base_pause_duration - steps_faster * self.pause_speedup_per_round,
    )
```

Change 3: call it from start_next_round

Add one line right after the append:

```
python
self.sequence.append(new_color)
```

```
self.update_playback_speed()
```

It has to come after the append, because the round number is the sequence length. reset() clears the sequence and then calls start_next_round(), so pressing R automatically goes back to the slow speed with no extra code.

What speeds you'll get

Round Flash (ms) Pause (ms)

1 450 200

3 380 170

6 275 125

10 135 65

11+ 120 (min) 60 (min)

Test it

Run python main.py and play 5 or 6 rounds. Each round's flashing should be visibly quicker than the last. If the change feels too slow or too fast, adjust flash_speedup_per_round and pause_speedup_per_round.

Commit it

```
git add game/game_engine.py
```

```
git commit -m "Task 2: add dynamic playback acceleration"
```

Tell me when it works, and we'll do Task 3, the sound effects.

I'll read the repo first so the Task 3 changes match your actual code.

Read 2 pages

Read 2 pages

I read the repo README, and it matches your handout. I can't open the code files, though, because GitHub blocks automated access to them, so I haven't seen your `game_engine.py`. I also don't see the PDF itself. The README plus the Task 2 notes you pasted are enough to plan from, but the Task 3 and Task 4 code needs to fit your real file.

Where you are: Tasks 1 and 2 are done (bug fixed, acceleration added). Tasks 3 and 4 are left:

- **Task 3:** each of the four pads gets its own pitch, played when it flashes during playback and when you click it.
- **Task 4:** a countdown bar during the player's turn. If time runs out before the pattern is finished, the game goes to `GAME_OVER`.

Step 1: paste your current code (you do this). Please paste these two files here, in their current state after Task 2:

1. `game/game_engine.py`
2. `game/color_button.py`

I'll start with Task 3, since sound hooks into the same places the flash logic already uses.

Step 2: install numpy (after you paste the files). Tones are generated in code, so no audio files are needed. With your venv active:

```
pip install numpy
```

Pygame's `pygame.sndarray` needs numpy to turn a generated wave into a playable sound.

Step 3: Task 3 together. Once I see your files, I'll give you:

1. A small tone generator, so each pad gets its own pitch (for example C4, E4, G4 and C5).
2. A place to build the four sounds once at startup, so there's no lag during play.
3. Two calls to play a pad's sound: one where playback flashes a pad, and one where a click registers.
4. A commit message: `Task 3: add per-pad audio tones`

Then we'll test and move on to Task 4 the same way.

Two quick things for later:

- **Chat export:** the handout wants the full chat history link, so keep working in this one conversation.
- **After video:** record it once Task 4 is done, so it shows the bug fixed, the speed-up, the sounds and the timer bar in one take.

Go ahead and paste the two files whenever you're ready.

Files hidden when shared

Tell, me how to go ahead with step 3 step by step.

Your code matches the Task 2 notes, so Task 3 is four small edits. One change to my earlier plan: we don't need numpy. Python's built-in `array` and `math` modules are enough, so there's nothing extra to install.

Step 1: Set up the audio format in `main.py`

Add one line before `pygame.init()`. It makes the audio format predictable (44.1 kHz, 16-bit, mono) so the tone generator works the same on every machine.

```
python
```

```
def main():  
    pygame.mixer.pre_init(44100, -16, 1, 512)  
    pygame.init()
```

Step 2: Add the tone generator in `game/game_engine.py`

Add these imports at the top of the file:

```
python
```

```
import math
```

```
import array
```

Then add this function between the imports and `class GameEngine`:

python

```
def make_tone(frequency, duration_ms=250, volume=0.4):
    """Build a short sine-wave beep as a pygame Sound (no audio files needed)."""
    sample_rate, _, channels = pygame.mixer.get_init()
    total = int(sample_rate * duration_ms / 1000)
    fade = int(sample_rate * 0.02) # 20 ms fade in/out so it doesn't click

    samples = array.array("h")
    for i in range(total):
        envelope = min(1.0, i / fade, (total - i) / fade)
        value = int(32767 * volume * envelope * math.sin(2 * math.pi * frequency * i /
sample_rate))
        for _ in range(channels):
            samples.append(value)

    return pygame.mixer.Sound(buffer=samples.tobytes())
```

Step 3: Build the four sounds once in `__init__`

Put this right after the `self.buttons = [...]` list:

python

```
# One pitch per pad: C4, E4, G4, C5
self.sounds = [make_tone(freq) for freq in (262, 330, 392, 523)]
```

They're built once at startup, so there's no lag during play.

Step 4: Play the sound in three places

a) `start_next_round` (the first pad of the replay):

python

```
self.buttons[self.sequence[0]].is_lit = True
self.sounds[self.sequence[0]].play()
```

b) `update`, inside the WATCH branch, where the next pad lights up:

python

```
self.buttons[next_id].is_lit = True
self.sounds[next_id].play()
```

c) `handle_event`, when the player clicks a pad:

python

```
btn.is_lit = True
self.sounds[btn.color_id].play()
```

That covers sound on flash during playback and on click during the player's turn.

Step 5: Test

Run `python main.py` and check:

- Each pad has a different pitch, both when the game flashes it and when you click it.
- The sound speeds up with the playback in later rounds.
- There are no crackles or clicks at the start or end of each note.

If you hear nothing, check your system volume. If you get a mixer error, tell me the message.

Step 6: Commit

```
git add game/game_engine.py main.py
git commit -m "Task 3: add per-pad audio tones"
```

Tell me once it works, or paste any error you hit. Then we'll do Task 4, the turn timer bar.

I didn't get from the step 4 part b and part c.

Here are parts b and c with the full surrounding code, so you can see exactly where each line goes.

Part b: inside the update method

Find this block in `update` (in the `else:` branch of the `WATCH` state):

Before:

`python`

```
else:
    if now - self.step_start_time >= self.pause_duration:
        self.showing_step += 1
        if self.showing_step < len(self.sequence):
            next_id = self.sequence[self.showing_step]
            self.buttons[next_id].is_lit = True
            self.is_flashing = True
            self.step_start_time = now
        else:
            self.state = "PLAYER_TURN"
```

After (one new line):

`python`

```
else:
    if now - self.step_start_time >= self.pause_duration:
        self.showing_step += 1
        if self.showing_step < len(self.sequence):
            next_id = self.sequence[self.showing_step]
            self.buttons[next_id].is_lit = True
            self.sounds[next_id].play()    # <-- NEW
            self.is_flashing = True
```

```

        self.step_start_time = now
    else:
        self.state = "PLAYER_TURN"

```

This is the moment the game lights the next pad in the sequence, so the sound plays right then. The first pad of each round is covered by part a.

Part c: inside the `handle_event` method

Find this block:

Before:

python

```

        if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
            for btn in self.buttons:
                if btn.contains(event.pos):
                    btn.is_lit = True
                    self.player_lit_button = btn
                    self.player_lit_start = pygame.time.get_ticks()

                    self.register_player_click(btn.color_id)
                    break

```

After (one new line):

python

```

        if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
            for btn in self.buttons:
                if btn.contains(event.pos):
                    btn.is_lit = True
                    self.sounds[btn.color_id].play() # <-- NEW
                    self.player_lit_button = btn
                    self.player_lit_start = pygame.time.get_ticks()

                    self.register_player_click(btn.color_id)
                    break

```

This is the moment the player clicks a pad. `btn.color_id` is 0 to 3, which matches the order of the `self.sounds` list (Red, Blue, Green, Yellow).

The pattern

Every time the code sets a pad's `is_lit = True`, it also plays that pad's sound. There are three such places in the file:

1. `start_next_round`: `self.sounds[self.sequence[0]].play()`
2. `update`: `self.sounds[next_id].play()`
3. `handle_event`: `self.sounds[btn.color_id].play()`

The only thing that changes is the index inside `self.sounds[...]`. It's whichever pad is being lit at that point.

Make these two edits, run `python main.py`, and tell me whether you hear a different pitch on each pad.

Next?

Assuming Task 3 works and you've committed it, here is Task 4. If you haven't committed yet, run `git add game/game_engine.py main.py` then `git commit -m "Task 3: add per-pad audio tones"` first, so each task keeps its own commit.

The idea

When the player's turn starts, we record the time and give them a time limit. Longer sequences get more time:

```
time_limit = 3000 ms + 1000 ms × (number of steps)
```

So round 1 gives 4 seconds, round 5 gives 8 seconds, and so on. Every frame, we check whether time ran out (game over) and draw a bar that shrinks as time runs down.

There are five edits, all in `game/game_engine.py`.

Edit 1: timer settings in `__init__`

Add this right after `self.player_flash_duration = 150`:

python

```
# Player turn timer (milliseconds)
self.turn_base_time = 3000
self.turn_time_per_step = 1000
self.turn_start_time = 0
self.turn_time_limit = 0
self.game_over_reason = ""
```

Edit 2: start the timer when the player's turn begins

In `update`, find `self.state = "PLAYER_TURN"` and add the two timer lines under it:

Before:

python

```
else:
    self.state = "PLAYER_TURN"
```

After:

python

```
else:
```

```

        self.state = "PLAYER_TURN"
        self.turn_start_time = now
        self.turn_time_limit = self.turn_base_time + self.turn_time_per_step *
len(self.sequence)

```

Edit 3: check for timeout every frame

Still in `update`, add this at the very end of the method. It should line up with the `if self.state == "WATCH":` line, not sit inside it:

python

```

if self.state == "PLAYER_TURN":
    if now - self.turn_start_time >= self.turn_time_limit:
        self.game_over_reason = "TIME'S UP! GAME OVER"
        self.state = "GAME_OVER"

```

Edit 4: set the reason for a wrong click

In `register_player_click`:

Before:

python

```

if self.player_input[current_idx] != self.sequence[current_idx]:
    self.state = "GAME_OVER"
    return

```

After:

python

```

if self.player_input[current_idx] != self.sequence[current_idx]:
    self.game_over_reason = "WRONG PATTERN! GAME OVER"
    self.state = "GAME_OVER"
    return

```

This lets the Game Over screen say whether you ran out of time or clicked wrong.

Edit 5: draw the bar and show the reason in render

a) The timer bar. Add this right after the `for btn in self.buttons:`

`btn.render(screen)` loop:

python

```

if self.state == "PLAYER_TURN":
    now = pygame.time.get_ticks()
    remaining = max(0, self.turn_time_limit - (now - self.turn_start_time))
    fraction = remaining / self.turn_time_limit

    bar_w, bar_h = 300, 18

```

```

        bar_x = self.width // 2 - bar_w // 2
        bar_y = 460

        if fraction > 0.5:
            bar_color = (80, 240, 130)
        elif fraction > 0.25:
            bar_color = (255, 220, 80)
        else:
            bar_color = (240, 70, 70)

        pygame.draw.rect(screen, (45, 48, 58), (bar_x, bar_y, bar_w, bar_h),
border_radius=9)
        pygame.draw.rect(screen, bar_color, (bar_x, bar_y, int(bar_w * fraction), bar_h),
border_radius=9)

```

The bar sits below the pads, at y=460. It shrinks from full to empty and turns green, then yellow, then red.

b) The Game Over text. Change this line:

Before:

python

```

over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70,
70))

```

After:

python

```

over_surf = self.font_title.render(self.game_over_reason, True, (240, 70, 70))

```

Test

Run `python main.py` and check these three things:

1. **Bar:** it appears when "Your turn" starts, shrinks smoothly, and changes color as it empties.
2. **Timeout:** wait without clicking and the screen should say "TIME'S UP! GAME OVER".
3. **Wrong click:** a wrong click should still say "WRONG PATTERN! GAME OVER".

Press R to restart. The timer resets on its own because the next player turn starts a fresh one.

Commit

```

git add game/game_engine.py
git commit -m "Task 4: add player turn countdown timer"

```

If the bar feels too tight or too generous, change `turn_base_time` and `turn_time_per_step` in Edit 1.

After that, the only things left are the **after video** (show the fixed bug, the speed-up, the sounds and the timer in one 10-second take) and the **chat link** for submission. Let me know how the test goes, or paste any error you see.

I did not commit task 3 yet.

No problem. Which situation you're in depends on whether you've started the Task 4 edits.

Case A: you haven't touched Task 4 yet (the usual case)

Commit Task 3 now, then continue with Task 4:

```
git status
git add game/game_engine.py main.py
git commit -m "Task 3: add per-pad audio tones"
```

`git status` should list only `game_engine.py` and `main.py` as modified. If it also lists `venv/` or `__pycache__/,` your `.gitignore` is missing or wrong. Fix that first, and don't use `git add ..`

Then apply the Task 4 edits and commit them separately.

Case B: you already pasted the Task 4 edits into the file

The two tasks are now mixed in the same file. You have two options.

Option 1 (simplest): finish and test Task 4, then split the commit with interactive staging:

```
git add -p game/game_engine.py
```

Git shows the changes in chunks. Answer `y` for the Task 3 chunks (`make_tone`, the `import math/import array` lines, `self.sounds`, the `.play()` lines) and `n` for the Task 4 chunks (timer settings, the timeout check, the bar drawing, `game_over_reason`). Then:

```
git add main.py
git commit -m "Task 3: add per-pad audio tones"
git add game/game_engine.py
git commit -m "Task 4: add player turn countdown timer"
```

Chunks that sit next to each other may be grouped together. Type `s` to split them. If this feels fiddly, use Option 2.

Option 2 (acceptable fallback): commit everything together as one commit:

```
git add game/game_engine.py main.py
git commit -m "Task 3 and 4: add audio tones and turn timer"
```

The handout asks for a separate commit per task, so Option 1 is better if you can manage it. Either way, the submission is the videos and the chat link.

Tell me which case you're in, and whether the Task 3 sounds played correctly when you tested, since that's the thing to confirm before committing.

Files hidden when shared

Task 3 is committed. The line `[main 45d0110] Task 3: add per-pad audio tones with 2 files changed` is exactly what we wanted. Your history now has three separate commits: Task 1, Task 2 and Task 3.

The two untracked items are fine and were correctly left out of the commit: `Lab-4/` (your before video) and `game/__pycache__/`. To stop `__pycache__` showing up in `git status`, add a `.gitignore` now:

```
printf "venv\nse_venv\n__pycache__\n" > .gitignore
git add .gitignore
git commit -m "Add .gitignore"
```

I included `se_venv/` because that's your `venv`'s name. It's outside the project folder, so it's harmless either way.

Task 4 is next

Go back to my Task 4 message and apply the five edits to `game/game_engine.py`:

1. Timer settings in `__init__`
2. Start the timer when `state` becomes `"PLAYER_TURN"`
3. The timeout check at the end of `update`
4. `game_over_reason` for a wrong click
5. The timer bar and the Game Over text in `render`

Then test:

- The bar appears and shrinks during your turn.
- If you wait without clicking, you get `"TIME'S UP! GAME OVER"`.
- A wrong click still says `"WRONG PATTERN! GAME OVER"`.

Then commit:

```
git add game/game_engine.py
git commit -m "Task 4: add player turn countdown timer"
```

If you get stuck on any of the five edits, paste your `game_engine.py` or the error here and I'll point to the exact line.

After Task 4

1. Record the **after video** (about 10 seconds). Show the sequence growing by one step, the speed-up, the sounds, and the timer bar, ideally ending in a timeout.
2. Copy it into `Lab-4/` as `after.mp4`.
3. Push your commits:

```
git push origin main
```

4. Submit the two videos and the link to this chat.

Files hidden when shared

I did not understand this.

Sorry, that was a lot of terminal syntax at once. That step is **optional** and has nothing to do with the game, so you can skip it and go straight to Task 4 if you prefer.

What it's for

When you run the game, Python creates a folder called `game/__pycache__` with auto-generated files. Git shows it as "untracked" every time you run `git status`, and you could commit it by accident with `git add ..`. A `.gitignore` file tells git to ignore certain files and folders.

What the command does

```
printf "venv/\nse_venv/\n__pycache__/\n" > .gitignore
```

It creates a text file called `.gitignore` containing three lines:

```
venv/  
se_venv/  
__pycache__/
```

- `printf "..."` prints the text. `\n` means "new line".
- `> .gitignore` saves that text into a file named `.gitignore`.

Easier way, if the command confuses you

1. Open the project folder in VS Code (or any text editor).
2. Create a new file named exactly `.gitignore` (with the dot at the start).
3. Paste those three lines into it and save.

Then the two git commands

```
git add .gitignore  
git commit -m "Add .gitignore"
```

The first stages the new file and the second saves it as a commit. After that, `git status` will stop listing `__pycache__`.

Where you are

Task 3 is already committed, so you're fine either way. The next thing is **Task 4** (the five edits to `game/game_engine.py` from my earlier message). Do you want to go through those one edit at a time?

Let us go to task 4

We'll do Task 4 in small chunks, and I'll show where each edit goes in your file. Open `game/game_engine.py`.

What we're building

When the player's turn starts, we record the time and give them a limit. Longer sequences get more time:

`time limit = 3 seconds + 1 second per step in the sequence`

Every frame, the game checks whether time has run out. If it has, it's game over. A bar below the pads shrinks as the time runs down.

Edit 1: add the timer settings in `__init__`

Find these lines:

python

```
self.player_lit_button = None
self.player_lit_start = 0
self.player_flash_duration = 150
```

Add the new block directly under `self.player_flash_duration = 150`:

python

```
self.player_lit_button = None
self.player_lit_start = 0
self.player_flash_duration = 150

# Player turn timer (milliseconds)
self.turn_base_time = 3000
self.turn_time_per_step = 1000
self.turn_start_time = 0
self.turn_time_limit = 0
self.game_over_reason = ""
```

These are just starting values. `turn_start_time` and `turn_time_limit` get filled in when the player's turn begins. `game_over_reason` will hold the text shown on the Game Over screen.

Edit 2: start the timer when the player's turn begins

In the `update` method, find the place where the state changes to `"PLAYER_TURN"`:

Before:

python

```
else:
    self.state = "PLAYER_TURN"
```

After:

python

```
        else:
            self.state = "PLAYER_TURN"
            self.turn_start_time = now
            self.turn_time_limit = self.turn_base_time + self.turn_time_per_step *
len(self.sequence)
```

The two new lines are indented to line up with `self.state = ...` now already exists in `update` (it's `pygame.time.get_ticks()` at the top of the method), so you don't need to define it.

Check before moving on

Run `python main.py`. Nothing should look different yet, because we haven't drawn the bar or added the timeout. The game should still play exactly as before, with sounds. If you get an error, paste it here.

Tell me when that works and we'll do Edits 3 and 4 (the timeout check and the Game Over reason).

Yeah, done.

Next are Edits 3 and 4: the timeout check and the Game Over reason.

Edit 3: check for timeout every frame

This goes at the **very end** of the `update` method, after the whole `if self.state == "WATCH":` block. It must line up with that `if` (the same indentation, 8 spaces), not sit inside it.

Before (the end of your `update` method):

python

```
        else:
            self.state = "PLAYER_TURN"
            self.turn_start_time = now
            self.turn_time_limit = self.turn_base_time + self.turn_time_per_step *
len(self.sequence)
```

After:

python

```
        else:
            self.state = "PLAYER_TURN"
            self.turn_start_time = now
            self.turn_time_limit = self.turn_base_time + self.turn_time_per_step *
len(self.sequence)

        if self.state == "PLAYER_TURN":
            if now - self.turn_start_time >= self.turn_time_limit:
                self.game_over_reason = "TIME'S UP! GAME OVER"
                self.state = "GAME_OVER"
```

Every frame, if it's the player's turn, the game checks whether the elapsed time has passed the limit. If it has, the game ends and records why.

Edit 4: set the reason for a wrong click

In `register_player_click`, find:

Before:

python

```
if self.player_input[current_idx] != self.sequence[current_idx]:
    self.state = "GAME_OVER"
    return
```

After:

python

```
if self.player_input[current_idx] != self.sequence[current_idx]:
    self.game_over_reason = "WRONG PATTERN! GAME OVER"
    self.state = "GAME_OVER"
    return
```

Now both ways of losing set a reason, and Edit 5 will display it.

Check before moving on

Run `python main.py`.

- **Wait without clicking** during your turn. After a few seconds the game should go to the Game Over screen (round 1 gives you 4 seconds).
- The screen will still say "WRONG PATTERN! GAME OVER" even for a timeout. That's expected, and Edit 5 fixes it.
- Press R and check that the game restarts normally.

If the game never ends on timeout, the usual cause is that the new block was indented inside the `if self.state == "WATCH":` block. Paste your `update` method here and I'll check it.

Tell me when this works and we'll do Edit 5: the shrinking bar and the correct Game Over text.

Files hidden when shared

Is the `if self.state == "PLAYER_TURN":` block correctly placed?

Yes, the `if self.state == "PLAYER_TURN":` block is in the right place. In your screenshot it starts at line 131, lined up with the `if self.state == "WATCH":` on line 110, so it runs every frame as intended. The `else:` on line 126 and the three timer lines under it are also correct.

A bug on line 123

Line 123 currently reads:

python

```
self.buttons[next_id].play = True
```

It should be two lines:

python

```
self.buttons[next_id].is_lit = True
self.sounds[next_id].play()
```

It looks like `.is_lit` was changed to `.play` by mistake while adding the sound in Task 3. As written, the pads after the first one won't light up during playback, and no sound plays for them either.

Fix it

1. On line 123, replace that one line with the two lines above. Keep the same indentation (the 16-space level, lined up with `next_id` on line 122).
2. Run `python main.py` and check that every pad in the sequence lights up with its own sound during playback.

About the commit

You already committed Task 3. If that commit contained the `.play = True` line, commit the fix separately before we continue:

```
git add game/game_engine.py
git commit -m "Fix: light pad and play sound during playback"
```

If you've already added the Task 4 edits to the same file, that command will include them in this commit. In that case, don't worry about it. Just finish Task 4 and commit it with the next step.

Tell me once the playback lights and sounds work, then we'll do Edit 4 (the `game_over_reason` for a wrong click) and Edit 5 (the shrinking bar).

Next edit.

Edit 4: record why the game ended on a wrong click

In `register_player_click`, find this block:

Before:

python

```
if self.player_input[current_idx] != self.sequence[current_idx]:
    self.state = "GAME_OVER"
    return
```

After:

python

```

if self.player_input[current_idx] != self.sequence[current_idx]:
    self.game_over_reason = "WRONG PATTERN! GAME OVER"
    self.state = "GAME_OVER"
    return

```

Only one line is new: `self.game_over_reason = "WRONG PATTERN! GAME OVER"`. Put it right above `self.state = "GAME_OVER"`, at the same indentation (12 spaces).

Both ways of losing now store a reason: the timeout in `update` (Edit 3) sets "TIME'S UP!", and a wrong click sets "WRONG PATTERN!". Edit 5 will show whichever one applies.

Check

Run `python main.py` and test both ways of losing:

1. **Click a wrong pad.** You should get the Game Over screen.
2. **Wait without clicking.** After about 4 seconds in round 1, you should get the Game Over screen.

The screen will still say "WRONG PATTERN! GAME OVER" in both cases for now, because the text in `render` is still fixed. That's expected, and Edit 5 fixes it. Also confirm that all pads light up with sound during playback, which was the line 123 fix.

Tell me when both cases end the game, and we'll do Edit 5: the shrinking timer bar and the correct Game Over text.

Next.

Edit 5: draw the timer bar and show the right Game Over text

Both changes are in the `render` method.

Part a: the timer bar

Find the loop that draws the buttons:

python

```

for btn in self.buttons:
    btn.render(screen)

```

Add this block directly **under** it, lined up with the `for` (8 spaces):

python

```

for btn in self.buttons:
    btn.render(screen)

if self.state == "PLAYER_TURN":
    now = pygame.time.get_ticks()
    remaining = max(0, self.turn_time_limit - (now - self.turn_start_time))
    fraction = remaining / self.turn_time_limit

    bar_w, bar_h = 300, 18

```

```

bar_x = self.width // 2 - bar_w // 2
bar_y = 460

if fraction > 0.5:
    bar_color = (80, 240, 130)
elif fraction > 0.25:
    bar_color = (255, 220, 80)
else:
    bar_color = (240, 70, 70)

pygame.draw.rect(screen, (45, 48, 58), (bar_x, bar_y, bar_w, bar_h),
border_radius=9)
pygame.draw.rect(screen, bar_color, (bar_x, bar_y, int(bar_w * fraction), bar_h),
border_radius=9)

```

What it does:

- `remaining` is how many milliseconds are left in the turn.
- `fraction` is that as a number from 1.0 (full) down to 0.0 (empty).
- The first `draw.rect` draws the dark background track. The second draws the colored bar on top, with its width multiplied by `fraction`, so it shrinks.
- The color goes green, then yellow, then red as time runs down.
- The bar is at `y = 460`, below the pads, which end around `y = 438`. The window is 520 pixels tall, so it fits.

Part b: the Game Over text

Find this line near the bottom of `render`:

Before:

python

```

over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70,
70))

```

After:

python

```

over_surf = self.font_title.render(self.game_over_reason, True, (240, 70, 70))

```

Instead of fixed text, it now shows whichever reason was stored (wrong click or time's up).

Test

Run `python main.py` and check:

1. **Bar appears:** when "Your turn" starts, a green bar shows under the pads and shrinks smoothly.
2. **Color changes:** it turns yellow, then red as it empties.
3. **Timeout:** wait without clicking and you get "TIME'S UP! GAME OVER".
4. **Wrong click:** a wrong click gives "WRONG PATTERN! GAME OVER".

5. **Restart:** pressing R starts round 1 again with a fresh bar.

Commit

Once everything works:

```
git add game/game_engine.py
git commit -m "Task 4: add player turn countdown timer"
```

Tell me how the test goes, or paste any error. After that, we'll do the **after video** and the push.

Next.

Everything is in place, so what's left is wrapping up. I'm assuming the test passed. If anything on the checklist failed, tell me which item and we'll fix it before moving on.

Step 1: confirm the Task 4 commit

```
git status
git log --oneline
```

You should see four commits, one per task, with the line 123 fix either in its own commit or folded into one of them:

```
Task 4: add player turn countdown timer
Task 3: add per-pad audio tones
Task 2: add dynamic playback acceleration
Task 1: fix sequence duplication bug
```

If `git status` still shows `game_engine.py` as modified, the Task 4 commit hasn't happened yet:

```
git add game/game_engine.py
git commit -m "Task 4: add player turn countdown timer"
```

Step 2: record the after video (about 10 seconds)

On Ubuntu, press **Ctrl + Alt + Shift + R** to start and stop the built-in screen recorder. The file is saved in your **Videos** folder. Use whatever you used for the before video if you prefer.

In one take, show these things:

1. The game running and the pattern growing by **one step** per round.
2. The flashes visibly **speeding up** in later rounds.
3. The **timer bar** shrinking during your turn. Keep the sound on, because the video should have the audio tones.
4. Optionally, a timeout, so the "TIME'S UP! GAME OVER" screen appears.

To get to the speed-up quickly, play correctly through four or five rounds. The timer bar and timeout are easiest to show at the end.

Step 3: save it in the Lab-4 folder

```
cp ~/Videos/<your-file-name>.mp4 Lab-4/after.mp4
ls Lab-4
```

Replace <your-file-name> with the real name. `ls Lab-4` should show `before.mp4` and `after.mp4`.

Step 4: push your commits

```
git push origin main
```

This sends the four commits to your own fork (SwazRoot/58_color_pattern). It does not touch the original SETAPESU26 repo. If git asks you to log in, use your GitHub username and a personal access token as the password.

The videos in `Lab-4/` are for you to submit, so you don't have to commit them to the repo.

Step 5: submit

According to the README, submission is three things:

1. The before video (`before.mp4`)
2. The after video (`after.mp4`)
3. The link to this chat, with the complete history. Use the share option on this conversation and copy the link.

When you've done Step 1, tell me what `git log --oneline` shows, and I'll check that it's what the handout expects.

Files hidden when shared

Your history looks right. `git log --oneline` shows the four task commits plus the playback fix, all on top of the original repo commits:

```
9798449 Task 4: add player turn countdown timer
28ef58d Fix: light pad and play sound during playback
45d0110 Task 3: add per-pad audio tones
bd08f1b Task 2: add dynamic playback acceleration
32ace4a Task 1: fix sequence duplication bug
73aecad (origin/main) Update game_engine.py <- original repo
```

`git status` shows nothing modified, only the untracked `Lab-4/` and `__pycache__/`, so all your code is committed. One small note: the "Fix" commit has 16 insertions, so it probably picked up some Task 4 edits that were already in the file when you committed. That's harmless, and the final code is what counts.

Next: record the after video

1. Run `python main.py`.

2. Press **Ctrl + Alt + Shift + R** to start the screen recorder (a red dot shows in the top bar).
3. In about 10 seconds, show:
 - the pattern growing by one step per round,
 - the flashes getting faster,
 - the timer bar shrinking on your turn (audio on, so the tones are heard),
 - ideally a timeout ending with "TIME'S UP! GAME OVER".
4. Press **Ctrl + Alt + Shift + R** again to stop. The file lands in ~/Videos.

To reach the speed-up quickly, play correctly through four or five rounds. Then stop clicking at the end to show the timeout.

Then copy it and push

```
ls ~/Videos
cp ~/Videos/<your-file-name>.webm Lab-4/after.webm
git push origin main
```

Ubuntu's recorder usually saves `.webm`. If the handout needs `.mp4`, tell me and I'll give you a conversion command. Replace `<your - file - name>` with the real name shown by `ls`.

The push sends your five commits to your fork, `SwazRoot/58_color_pattern`. If it asks for a password, use a GitHub personal access token, not your account password.

Submit

Submit these three things:

1. `before.mp4`
2. `after.webm` (or `.mp4`)
3. The link to this chat. Use the share button on this conversation and copy the link.

Tell me once the video is recorded. If the push fails, paste the error and I'll sort it out.

Files hidden when shared

This is the video can u trim it and tell me what to do next?

Ran 3 commands, read 2 files

Ran 3 commands, read 2 files

I'm checking the last 16 seconds of your recording to pick the best 10-second clip.

Read a file, ran a command, shared files

Read a file, ran a command, shared files

I trimmed your 92-second recording to a 10-second clip, `after.mp4`, from 0:79 to 0:89. It shows:

- The score going from 4 to 5 to 6, with the pattern growing one step per round.
- The timer bar during "Your turn", shrinking each time.
- The Game Over screen at the end.

The recording has no audio track. Ubuntu's built-in screen recorder doesn't capture system sound, so the pad tones from Task 3 can't be heard in the video. Since the handout asks for the new features to be visible in the after video, you have two options:

1. **Re-record with sound.** Install OBS (`sudo apt install obs-studio`), add a "Screen Capture" source and an "Audio Output Capture" source, and record 10 seconds. I can trim that one the same way if you upload it.
2. **Submit this clip** and mention in the chat link that the sounds work but the recorder didn't capture audio. This is the weaker choice, and it's your call whether your instructor will accept it.

Timeout isn't shown either. The clip ends with "WRONG PATTERN!", not "TIME'S UP!". If you re-record, wait without clicking at the end to show the timeout.

Next steps

1. Download `after.mp4` from the file card above and copy it into the project's **Lab - 4** folder (or use the re-recorded one).
2. Push your commits:

```
git push origin main
```

3. Submit `before.mp4`, `after.mp4` and the share link to this chat.

Do you want to re-record with audio, or go ahead with this clip?

After

MP4

Yeah, i installed obs studio.

OBS can capture both the game window and its sound. Here is the setup, one step at a time.

Step 1: set the recording format

1. In OBS, go to **File** → **Settings** → **Output**.
2. Set **Recording Format** to mp4 (the default is often mkv).
3. Click **OK**.

Step 2: add the screen source

1. In the **Sources** box at the bottom, click +.
2. On Ubuntu with Wayland (the default), choose **Screen Capture (PipeWire)**. If you're on X11, choose **Window Capture (Xcomposite)** instead.
3. Name it and click **OK**.
4. In the popup, select your screen or the game window, then click **Share**.
5. The game should now appear in OBS's preview.

Step 3: add the audio source

1. Click + in Sources again and choose **Audio Output Capture (PulseAudio)**. It may also be called "Audio Output Capture (PipeWire)".
2. Pick your speakers or headphones as the device, then click **OK**.

Step 4: test the audio before recording

1. Run `python main.py` and click a pad.
2. In OBS's **Audio Mixer**, the new audio bar should jump when a tone plays.
3. If it doesn't move, check that you picked the right output device in Step 3, and that your system volume isn't muted.

Step 5: record

1. Click **Start Recording** in the bottom right.
2. Click on the game window so it has focus.
3. In about 10 seconds, show:
 - the pattern growing, with the tones audible,
 - the flashes speeding up,
 - the timer bar shrinking,
 - ideally a timeout at the end. Wait without clicking until "TIME'S UP! GAME OVER" appears.
4. Click **Stop Recording**.

To save time, you can play a few rounds first, then start recording from a later round. The speed-up is easiest to show in rounds 4 to 6.

Step 6: find the file

OBS saves recordings in your **Videos** folder by default. Check with:

`ls ~/Videos`

Upload the new file here and I'll trim it to 10 seconds and check that it has audio. If any step doesn't match what you see on your screen, tell me which one.

Can you give me an easier method instead of obs?

Yes. The easiest option is to use your phone.

Easiest: film the laptop screen with your phone

1. Open the game on your laptop and turn the laptop volume up.
2. Open your phone's camera in video mode and point it at the laptop screen, so the game window and the speakers are both in range.
3. Start recording, then play for about 10 seconds:
 - Let the pattern grow so the tones are audible.

- Keep playing so the flashes speed up and the timer bar shrinks.
 - End by waiting without clicking until "TIME'S UP! GAME OVER" appears.
4. Stop recording and send the video to your laptop (email, Google Drive, USB or a cable).
 5. Upload it here and I'll trim it to 10 seconds.

It's not as sharp as a screen recording, but it captures both the picture and the sound.

Alternative: Kazam (a simple screen recorder with audio)

```
sudo apt install kazam
```

1. Open Kazam and set **Screencast** mode.
2. Under **Sound**, enable **Speakers** (the option for sound from the computer).
3. Click **Capture**, play the game for 10 seconds, then stop from the top-bar icon.

Kazam can be unreliable on Wayland. If the screen comes out black, use the phone method.

What to show either way

- The sequence growing by one step per round, with different tones per pad.
- The flashes getting faster.
- The timer bar shrinking, ideally ending in a timeout.

Upload the new recording whenever it's ready.

Files hidden when shared

This is the video.

Ran 3 commands, read 2 files

Ran 3 commands, read 2 files

I'm looking at one full-size frame to decide how to crop the clip so the game is easier to see.

Read 2 files, ran a command, shared files

Read 2 files, ran a command, shared files

I've trimmed your phone video to a 10-second clip, `after_with_sound.mp4`, cropped to the game window so it's easier to see. I picked the stretch from 0:58 to 1:08 of your recording. The audio track is there and the tones register clearly.

The clip shows:

- Rounds at scores 5 and 6, with the pattern flashing step by step.
- The timer bar shrinking during "Your turn".
- A different tone on each pad.
- The Game Over screen at the end.

Two limits:

1. The video ends with "WRONG PATTERN! GAME OVER", not "TIME'S UP!", so the timeout isn't shown. If your instructor wants to see the timeout specifically, record a short extra take where you wait without clicking. Otherwise this clip covers the other features.
2. I couldn't listen to the audio, only measure it. The level shows tones are present. Play the clip once yourself and check that you hear different pitches.

Next steps

1. Download `after_with_sound.mp4` from the card above.
2. Copy it into your project folder as `after.mp4`:

```
cp ~/Downloads/after_with_sound.mp4 ~/se_project/58_color_pattern/Lab-4/after.mp4
ls ~/se_project/58_color_pattern/Lab-4
```

You should see `before.mp4` and `after.mp4`.

3. Push your commits:

```
git push origin main
```

4. Submit `before.mp4`, `after.mp4` and the share link to this chat.

If the push asks for a password, use a GitHub personal access token, not your account password. Tell me if the push fails and paste the error.

After with sound
MP4

Files hidden when shared

The push failed because GitHub no longer accepts your account password for git. The error says "Password authentication is not supported for Git operations." You need a **personal access token**, which you paste in place of the password. Your commits are safe locally, so nothing is lost.

Step 1: create the token

1. Log in to github.com in your browser.
2. Click your profile picture, then **Settings**.
3. Scroll to the bottom of the left menu and click **Developer settings**.
4. Click **Personal access tokens**, then **Tokens (classic)**.
5. Click **Generate new token**, then **Generate new token (classic)**.
6. Give it a name such as `lab4-push`, set an expiration (7 days is enough), and tick the **repo** checkbox.
7. Click **Generate token** at the bottom.
8. **Copy the token now**. It starts with `ghp_`, and GitHub shows it only once.

Step 2: push again

```
git push origin main
```

When it asks:

- **Username:** SwaZRoOT
- **Password:** paste the token. The terminal shows nothing while you paste, which is normal. Press Enter.

You should see lines like `Writing objects: 100%` and `main -> main`.

Step 3: check it worked

Open https://github.com/SwaZRoOT/58_color_pattern in your browser. You should see your four task commits, plus the fix commit, at the top of the history.

Notes

- Treat the token like a password. Don't paste it into this chat or commit it anywhere.
- If you want git to remember it so it doesn't ask again, run `git config --global credential.helper store` before pushing. It saves the token in plain text on your laptop, so skip this if the laptop is shared.
- Lab-4/ (the videos) isn't committed, so it won't appear on GitHub. That's fine, because the videos are submitted separately.

Tell me whether the push succeeded or paste any new error.

Files hidden when shared

Good news first: your token worked. The login succeeded this time, and the new error is a different problem. GitHub rejected the push with `fetch first`, which means your fork on GitHub has a commit that your laptop doesn't have yet. Your local commits are still safe.

Step 1: see what's on GitHub that you don't have

```
git fetch origin
git log --oneline HEAD..origin/main
```

This lists the commits that exist only on GitHub. It's probably one small commit, such as a README edit or a file change made in the browser. Paste the output here if you're not sure what it is.

Step 2: bring it into your local copy

```
git pull --no-rebase origin main
```

This merges the GitHub commit with your five commits. Git may open a text editor asking for a merge message:

- In **nano**: press **Ctrl + O**, then **Enter**, then **Ctrl + X**.
- In **vim**: type `:wq` and press **Enter**.

Step 3: push again

```
git push origin main
```

Enter SwaZRoOT as the username and paste your token as the password, like last time.

If Step 2 reports a conflict

You'd see **CONFLICT (content): Merge conflict in game/game_engine.py**. That would mean the GitHub commit changed the same lines you edited. Don't panic, and don't try to fix it by guessing. Run `git status` and paste the output here. I'll walk you through it.

Don't do this

Don't use `git push --force` to get past the error. It would overwrite whatever is on GitHub, and you could lose work.

Tell me what Step 1 shows, or whether the pull and push went through.

Files hidden when shared

The fetch worked, and the two commits on GitHub are harmless:

```
2111ace Delete before.mp4
94a84e7 Add files via upload
```

It looks like a `before.mp4` was uploaded through GitHub's web page and then deleted. Those commits don't touch your code, so merging them won't cause a conflict.

Step 1: merge them into your local copy

```
git pull --no-rebase origin main
```

Git may open a text editor asking for a merge commit message. Just accept the default:

- In **nano**: press **Ctrl + O**, then **Enter**, then **Ctrl + X**.
- In **vim**: type `:wq` and press **Enter**.

You should see something like `Merge made by the 'ort' strategy`.

Step 2: push

```
git push origin main
```

Enter SwaZRoOT as the username and paste your token as the password.

Step 3: check it worked

1. The terminal should show `main -> main` with no rejected message.
2. Open https://github.com/SwaZRoOT/58_color_pattern in your browser. The latest commits should include your Task 1 to Task 4 commits and the merge commit.

Your local `Lab-4/` folder with `before.mp4` and the after video is untracked, so the pull won't touch it.

Tell me whether the pull and push went through, or paste any error.

Files hidden when shared

The push worked. The last line, `2111ace..306a2ae main -> main`, shows your commits landed on GitHub, with no `rejected` message this time. The merge also went through cleanly.

Quick check

Open https://github.com/SwaZRooT/58_color_pattern in your browser. The commit history should show:

- Task 1: fix sequence duplication bug
- Task 2: add dynamic playback acceleration
- Task 3: add per-pad audio tones
- Fix: light pad and play sound during playback
- Task 4: add player turn countdown timer
- the merge commit

What's left: submission

The handout lists three things:

1. **Before video:** `before.mp4`, which is in your `Lab-4/` folder.
2. **After video:** the clip I trimmed, `after_with_sound.mp4`. Make sure it's in `Lab-4/` as `after.mp4`. You can check with `ls Lab-4`.
3. **Chat link:** the share link to this conversation, with the complete history.

To get the chat link:

1. Click the **Share** button at the top of this conversation (on mobile it's in the menu).
2. Create the share link and copy it.
3. Open it in a private browser window to confirm it loads.

Tell the instructor's form or portal which link is which, if it asks.

One optional extra

The clip ends with "WRONG PATTERN!", so the "TIME'S UP!" screen isn't shown on video. If the grader may want to see the timeout, record a short extra clip of waiting out the timer and submit it too. If you don't have time, the current clip still shows the bar counting down during your turn.

Do you want to record the timeout clip, or are you ready to submit?

Now how to submit?

The README only says what to submit, not where, and I haven't seen the PDF handout, so I can't tell you which portal your course uses. Check the handout or your course page for a submission link (Google Form, LMS, email). Here is how to get the three items ready.

1. Get the chat link

1. Click the **Share** button at the top of this conversation. On mobile it's in the menu.
2. Create the link and copy it.
3. Open it in a private browser window to confirm the whole conversation loads. The handout wants the complete chat history.

If you send more messages after sharing, update the share link so it includes them.

2. Gather the videos

Check that both files are in your Lab - 4 folder:

```
ls ~/se_project/58_color_pattern/Lab-4
```

You should see `before.mp4` and `after.mp4`. Play each one once to confirm it works, and that the after video has sound.

If the portal asks for links instead of file uploads, upload the videos to Google Drive, set sharing to "Anyone with the link can view", and copy each link.

3. Submit the three things

1. `before.mp4`, the gameplay with the bug.
2. `after.mp4`, the bug fixed and the features working.
3. The share link to this chat.

If the handout also asks for your GitHub repo link, it's https://github.com/SwaZRo0T/58_color_pattern. It isn't in the README's checklist, but it doesn't hurt to include it.

Before you hit submit

- Is your token safe? Don't paste it anywhere. If you want to be careful, you can delete it in GitHub under Settings, Developer settings, Personal access tokens.
- Does the after video show the timeout? It's optional, but if you have time, one extra clip waiting out the timer covers it.

If you can find the submission instructions in the PDF, paste that part here and I'll tell you exactly what to do.

Files hidden when shared

The handout answers it, and it changes what I told you earlier. Videos **do** need to be in the repo. The handout says to upload all of these to your Lab repo under the Lab - 4 folder:

1. Videos (before and after)
2. Updated code
3. Chat history exported as a doc or PDF

Your code is already pushed with a separate commit per task, which is what the handout asks for. What's left is the videos and the chat PDF.

Step 1: export this chat as a PDF

1. Click **Share** at the top of this conversation and create the link.
2. Open the link in your browser.
3. Press **Ctrl + P**, choose **Save as PDF** as the destination, and save it as `chat_history.pdf`.
4. Check that the PDF includes the whole conversation.

Step 2: put everything in Lab - 4

```
cd ~/se_project/58_color_pattern
cp ~/Downloads/after_with_sound.mp4 Lab-4/after.mp4
cp ~/Downloads/chat_history.pdf Lab-4/chat_history.pdf
ls Lab-4
```

Change the paths if your files are somewhere else. `ls` should show `before.mp4`, `after.mp4` and `chat_history.pdf`.

Step 3: commit and push

Add only the Lab - 4 folder, so `__pycache__` stays out:

```
git add Lab-4
git commit -m "Add Lab-4 deliverables: videos and chat history"
git push origin main
```

Use your username and your token again when asked. Then open https://github.com/SwaZRooT/58_color_pattern and confirm the Lab - 4 folder shows the three files.

One thing to check: which repo is your "Lab repo"?

The handout says "your Lab repo" and "push to your repo in your personal account." If your course gave you a separate lab repository, copy the code there too, along with the Lab - 4 folder:

```
cp -r game main.py README.md <path-to-your-lab-repo>/Lab-4/
```

If the fork you've been pushing to is the repo your instructor expects, you're done after Step 3. Check the course page or ask your TA if you're not sure.

Also, the handout doesn't mention a chat link, only the exported doc or PDF. Include both if the portal allows it.

Tell me whether the push shows all three files on GitHub.

